

ДИСЦИПЛИНА	Программирование (полное наименование дисциплины без сокращений)
ИНСТИТУТ	Институт тонких химических технологий им. М.В. Ломоносова
КАФЕДРА	Кафедра информационных систем в химической технологии полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Практические занятия (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Серебренников Н.П. (фамилия, имя, отчество)
СЕМЕСТР	2-й семестр, 1-й курс бакалавриата, 2024-2025 учебный год (указать семестр обучения, учебный год)

Практическая работа № 1. Основы Python и использование Spyder IDE

Цель работы

Ознакомиться с основными компонентами среды разработки Spyder, а также понять и применить основы работы с изменяемыми и неизменяемыми типами данных, операторами id и is, а также ссылками на объекты в Python.

1. Открытие Spyder

Запустите Spyder. Ознакомьтесь с основными панелями:

- Editor — для написания кода.
- IPython Console — для выполнения кода.
- Variable Explorer — для просмотра переменных.
- File Explorer — для навигации по файлам.

Для этого нажмите на верхней панели управления Help -> Show tour

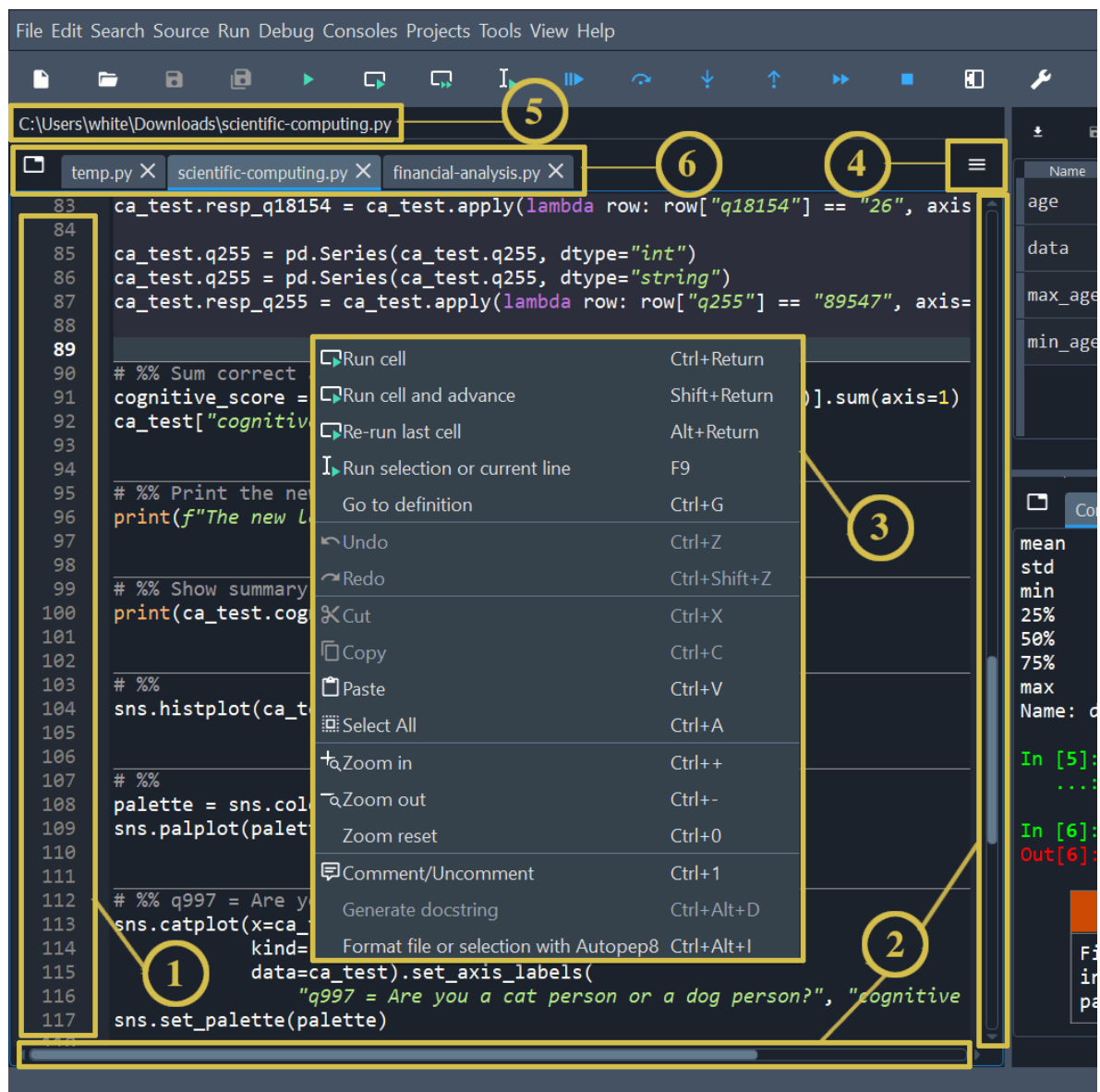


Рисунок 1. Панель эдатора.

- 1) Левая боковая панель показывает номера строк и отображает любые предупреждения после внутреннего анализа кода, которые существуют в текущем файле. Щелчок по номеру строки выбирает текст в этой строке, а щелчок справа от него устанавливает точку останова.
- 2) Полосы прокрутки позволяют осуществлять вертикальную и горизонтальную навигацию в файле.
- 3) Контекстное меню (щелчок правой кнопкой мыши) отображает действия, относящиеся к тому, что было нажато.
- 4) Меню параметров (значок «гамбургер» вверху справа) включает полезные настройки и действия, относящиеся к редактору.
- 5) Строка местоположения в верхней части панели редактора показывает полный путь к текущему файлу.
- 6) Панель вкладок отображает имена всех открытых файлов. Она также имеет кнопку «Обзор вкладок» (слева) для отображения каждой

открытой вкладки и переключения между ними, что удобно, если открыто много вкладок.

2. Создание первого скрипта

Напишите небольшой код, который выполняет следующие действия:

- Присваивает переменной `a` значение 5.
- Присваивает переменной `b` значение 10.
- Присваивает переменной `c` сумму `a` и `b`.
- Выводит результат.

Выполните код с помощью кнопки "Run" или горячих клавиш (например, F5).

Обратите внимание, как результат выводится в **IPython Console**, а переменные отображаются в **Variable Explorer**.

3. Использование Variable Explorer

После выполнения кода посмотрите на созданные переменные и их значения. Отметьте, как можно редактировать переменные непосредственно в **Variable Explorer**.

4. Откройте Help и найдите документацию по функции `print()`.

Прочитайте и кратко опишите её возможности в комментарии.

Для этого необходимо напечатать `print()` в эдиторе и нажать сочетание клавиш `Ctrl+I`.

5. Работа с переменными и типами данных.

1. Создайте переменные следующих типов:

- Целое число (`int`)
- Вещественное число (`float`)
- Строка (`str`)
- Булевый тип (`bool`)
- Список (`list`)
- Кортеж (`tuple`)
- Словарь (`dict`)
- Множество (`set`)

2. Используя функцию `type()`, выведите тип каждой переменной.

3. Используя **Variable Explorer**, проверьте значения и типы созданных переменных.

6. Ссылки на объекты и их поведение

Создание переменных и ссылки на объект

Проверьте, как ссылки на объекты работают для изменяемых типов:

```
list1 = [1, 2, 3]
list2 = list1
list2.append(4)
print("list1:", list1)
print("list2:", list2)
```

Почему list1 изменится, когда меняется list2? Ответ запишите в комментариях.

Работа с числовыми типами

В Python маленькие целые числа (например, от -5 до 256) могут кэшироваться, чтобы избежать лишнего использования памяти.

Проверьте поведение с такими числами:

```
x = 1000
y = 1000
print(x is y)  # Может быть False
```

Почему был выведен такой ответ в вашем коде?

7. Функция `id` и оператор `is`.

1. Создайте две переменные с одинаковыми значениями (например, две строки "hello").
2. Используя функцию `id`, выведите идентификаторы этих переменных. Объясните, почему они могут быть одинаковыми или разными.
3. Используя оператор `is`, проверьте, ссылаются ли переменные на один и тот же объект.
4. Повторите эксперимент с изменяемыми и неизменяемыми типами данных. Сделайте выводы. И запишите их в комментариях.

Пример:

```
hello = "hello"
he = "hello"
print(id(hello))
print(id(he))
print(he is hello)
```

8. Глубокие и поверхностные копии

Напишите программу, которая демонстрирует разницу между глубокими и поверхностными копиями:

```
original = [[1, 2], [3, 4]]
shallow_copy = original
deep_copy = [list(sublist) for sublist in original]
```

```
shallow_copy[0][0] = 100
print("Original:", original)
print("Shallow copy:", shallow_copy)
```

```
deep_copy[0][0] = 200
print("Deep copy:", deep_copy)
```

Опишите почему выведен подобный результат в комментарии.

9. Работа с кортежами.

1. Создайте кортеж и попытайтесь изменить его (например, добавьте элемент). Объясните результат.
2. Создайте два кортежа с одинаковыми значениями. Используя `id` и `is`, проверьте, ссылаются ли они на один и тот же объект.
3. Сравните поведение кортежей и списков в контексте изменяемости и ссылок на объекты.

10. Отладка в Spyder.

1. Установите точку останова (breakpoint) в вашем коде.
2. Запустите программу в режиме отладки (Debug).
3. Используя панель **Variable Explorer**, следите за изменением переменных в процессе выполнения программы.
4. Используя кнопки **Execute Current Line**, **Step Into** и **Execute until returns**, **Continue Execution until next breakpoint**, пройдите по коду шаг за шагом. Объясните, как работает каждая из этих функций.

В качестве простого примера можете использовать код:

```
delimiters = ["[", "]", "=", "-", "#", "/", "\\\"", "(", ")"]
for i in delimiters:
    print(i)
```

11. Итоговый эксперимент.

1. Создайте список и кортеж с одинаковыми значениями.
2. Используя функцию `id`, проверьте, ссылаются ли они на один и тот же объект.
3. Измените список и проверьте, как это повлияет на кортеж. Объясните результат.

Контрольные вопросы:

Напишите ответы в комментариях.

1. Какие панели Spyder вы использовали в ходе работы? Как они помогли вам выполнить задания?
2. В чём разница между изменяемыми и неизменяемыми типами данных?
3. Как функция `id` помогает понять работу с объектами в Python?
4. Почему оператор `is` может возвращать `False` для двух переменных с одинаковыми значениями?
5. Как работают ссылки на объекты в Python? Почему изменение одного списка может повлиять на другой?
6. Как работает отладка в Spyder? Какие инструменты вы использовали?

Дополнительно:

Напишите код, который принимает строку, представляющую целое число (например, `"123"`), и преобразует её в целое число. Затем увеличьте это число на 10 и выведите результат.

```
number_str = "42"
```

Напишите код, который принимает целое число (например, `100`) и преобразует его в строку. Затем добавьте к этой строке текст `" рублей"` и выведите результат.

```
number = 100
```

Преобразование списка чисел в строку

Напишите код, который принимает список целых чисел (например, `[1, 2, 3, 4, 5]`) и преобразует его в строку, где числа разделены запятыми. Выведите результат. Используйте `map` (<функция преобразования в строку>, <список>) и `join`.

Пример синтаксиса `join`:

`"разделитель".join(строка)`

```
numbers = [1, 2, 3, 4, 5]
```

Дополнительное самостоятельное задание (можете сделать дома).

Установка Python и настройка среды разработки на Windows 10.

Так как в дальнейшем, в лабораторных работах вам придется переключать ядро python, рекомендуем установить python и Spyder, а не отдельно Spyder со своей средой выполнения.

- 1) Скачайте выполняемый файл установщик python для Windows 10/11 с сайта: <https://www.python.org/downloads/windows/> (в классах установлен 3.12.8)
- 2) Запустите установку, следуйте рекомендациям.
- 3) Скачайте выполняемый файл установщик python для Windows 10/11 с сайта: <https://www.spyder-ide.org/download/>
- 4) Запустите установку, следуйте рекомендациям.
- 5) После установки, если вы не меняли настроек по умолчанию, Spyder запустится.

Установка Python и настройка среды разработки на Linux (Astra)

1. Установка Python в Debian подобной ОС

- Обновление системы.

Откройте терминал и выполните команду для обновления списка пакетов:

```
sudo apt update
```

Установите обновления для системы:

```
sudo apt upgrade
```

- Установка Python:

Установите Python с помощью пакетного менеджера:

```
sudo apt install python3
```

Проверьте установку Python, выполнив команду:

```
python3 --version
```

Убедитесь, что Python установлен корректно, и вы видите версию Python (например, `Python 3.x.x`).

- **Установка pip (менеджер пакетов Python):**

Установите `pip` для управления пакетами Python:

```
sudo apt install python3-pip
```

Проверьте установку `pip`:

```
pip3 --version
```

2. Установка Conda и Spyder

- **Скачайте установочный скрипт Miniconda**

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

- **Запустите установку**

```
bash Miniconda3-latest-Linux-x86_64.sh
```

- **Следуйте инструкциям на экране. При запросе согласия на добавление Conda в PATH, ответьте "yes".**

- **Проверка установки Conda:**

```
conda -version
```

- **Обновите Conda до последней версии:**

```
conda update conda
```

- **Установка Spyder:**

- **Установите Spyder с помощью Conda:**

```
conda install spyder
```

- **После завершения установки, запустите Spyder:**

```
spyder
```


3. Проверка установки и настройка окружения

- Проверка переменных окружения:

Выведите список переменных окружения, связанных с Python:

```
env | grep -i python
```

Найдите переменную `PYTHONPATH` и объясните её назначение.

- Проверка установки Spyder:

Запустите Spyder и убедитесь, что он открывается корректно

В терминале проверьте версию Spyder:

```
spyder --version
```